

Deep Learning Applications for Resource Optimization

Efficient training, compression, and deployment of neural networks

PRESENTED BY

Er. Abhishek Kumar Shukla

Senior AI Developer
UniConverge Technologies

DATE • TIME

30 July 2026

11:30 AM – 1:00 PM (90 min)

D L

Contents

Eight sections · Part A (1–6) conceptual, Part B (7–8) live demo and results



01

Why Resource Optimization Matters

The deployment gap

02

The Cost of Scale

Memory & compute reality

03

Efficient Training: **PEFT** & **LoRA**

Train less, not from scratch

04

Precision Formats

FP32 · FP16 · BF16 · INT8

05

Quantization

Compress for deployment

06

Model Export & **ONNX**

Portable deployment format

07

Live Demo: Two Models, One Journey

Hands-on in Google Colab

08

Live Results, Benchmarking & Takeaways

Verified measured numbers

PART 01 / 08

Why Resource Optimization Matters

The gap between what a model can do in a paper and what it can do on real, constrained hardware.



The Problem: Bigger Models, Harder Constraints

¶ 01 · THE HARDWARE GAP

Bigger Models vs. Static Hardware

State-of-the-art model sizes have grown from millions to hundreds of billions of parameters in under a decade.

Meanwhile, deployable target devices — laptops, phones, edge sensors, and single research GPUs — have not grown at the same rate, sharply widening the capability-hardware gap.

¶ 02 · THE PIVOTAL QUESTION

Designing for Deployability

"Can we make an already-good model small enough, fast enough, and cheap enough to actually deploy where it's needed?"

Every optimization technique in this session answers that core question from a different angle:

PEFT / LoRAQuantizationONNX Export

Three Resources Being Optimized

Every technique in this session trades one of these three levers.



RESOURCE 01

Memory

GPU/CPU RAM required to hold model weights, activations, and optimizer states. Often the hard limit on what can even be loaded onto a device — before speed or cost ever become a factor. If a model doesn't fit in memory, nothing else matters.



RESOURCE 02

Compute Time

Two distinct time budgets: training time (how long until a usable model exists) and inference latency (how fast a single prediction returns once deployed). Both cost real money and limit what users will tolerate.



RESOURCE 03

Energy / Cost

Cloud GPU-hours and power draw translate directly into money and, at scale, into a real carbon footprint. Optimization isn't just a technical preference — it determines what's sustainable to run continuously.

Two Different Optimization Moments

MOMENT 01

Training-Time Optimization

How do we adapt or build a model without needing the enormous compute budget that training from scratch would require? This is where **PEFT** and **LoRA** come in — freeze most of the pretrained model and train only a small targeted addition.

Tools → PEFT · LoRA · QLoRA

MOMENT 02

Deployment-Time Optimization

Once a model is trained, how do we shrink and speed it up for where it will actually run? This is where precision formats, quantization, and **ONNX** export come in — reduce bytes per parameter, then ship a portable runtime.

Tools → FP16/BF16 · INT8 · ONNX

Today's session covers both, in that order, then demonstrates each hands-on.

Setting Up the Rest of This Session

Every technique in this session answers the same question differently: **how do we get most of a large model's capability at a fraction of the resource cost?**

The five techniques that follow each attack a different part of the problem — training cost, numeric storage, runtime speed, or portability. Together they form a complete optimization stack.



STEP 01

Cost of Scale

Why full fine-tuning is expensive



STEP 02

PEFT / LoRA

Train a tiny fraction of params



STEP 03

Precision

Drop from 32-bit to 16-bit



STEP 04

Quantization

Compress to 8-bit or 4-bit



STEP 05

ONNX Export

Portable, optimized runtime

The Cost of Scale

SESSION OVERVIEW

What full fine-tuning actually costs in memory and compute, and why that cost motivated everything else in this session.

02

What 'Full Fine-Tuning' Actually Requires

Fine-tuning every parameter of a pretrained model means storing, for every weight: the weight itself, its gradient, and — for the **Adam** optimizer — two additional running-average terms per weight. Nothing here is optional; the optimizer simply does not work without all four pieces per parameter.

For a model with **N** parameters trained in 32-bit precision (**FP32**), full fine-tuning with **Adam** requires roughly 4 bytes (weight) + 4 bytes (gradient) + 8 bytes (**Adam**'s two moment terms), about 16 bytes per parameter, before even counting activation memory.

WEIGHT

The parameter itself, in FP32

4 B

GRADIENT

Computed during backprop

4 B

ADAM MOMENT 1

First running average

4 B

ADAM MOMENT 2

Second running average

4 B

= 16 BYTES PER PARAMETER • BEFORE ACTIVATION MEMORY

Memory Requirement Scales With Model Size

Full fine-tuning, **FP32**, with **Adam** — approximate GPU memory required.

~2 GB

125M-parameter model

Full fine-tuning, **FP32**. Fits comfortably on a single consumer GPU.

~24 GB

1.5B-parameter model

Already at the edge of a single consumer GPU — borderline on an RTX 4090 or A10.

~1 TB+

70B-parameter model

Requires a multi-GPU cluster, not a single machine.

*Same algorithm (**Adam**, **FP32**) — the model size, not the method, drives the memory cliff.*

Why This Motivates Everything Else Today

¶ 01 · THE GAP IS REAL

Most research groups, students, and even many companies do not have multi-GPU clusters sitting idle. A 1.5B-parameter model already pushes the edge of a single consumer GPU; a 70B-parameter model is simply out of reach without institutional-scale infrastructure.

The gap between "what the paper used" and "what we have on the desk" is real and persistent.

¶ 02 · THE BRIDGE FORWARD

This is the exact gap that Parameter-Efficient Fine-Tuning (**PEFT**) techniques — **LoRA** specifically — were built to close.

The next section explains how these methods bypass the memory wall by updating only a tiny fraction of the weights.

PART 03 · PEFT & LORA

Efficient Training: PEFT & LoRA

How to adapt a large pretrained model to a new task while updating a tiny fraction of its parameters.

03

Parameter-Efficient Fine-Tuning: The Core Idea

¶ 01 • FREEZE ALMOST EVERYTHING

Instead of updating all of a pretrained model's weights, **freeze almost everything** — and only train a small number of additional or selected parameters.

The pretrained model's general knowledge is preserved intact; only a small adapter is learned.

¶ 02 • MEMORY SHRINKS PROPORTIONALLY

If only **1-2% of parameters** are being trained, gradient and optimizer-state memory shrinks by roughly the same proportion.

*A model that needed **24 GB** for full fine-tuning can often be adapted on a single consumer GPU instead.*

MODEL PARAMS



TRAINABLE ~1-2%

Same model, same forward pass — only the gold portion is updated during fine-tuning.

Three Families of PEFT Techniques

Different ways to decide which small subset of parameters gets trained.

FAMILY 01

Additive Methods

Insert small new trainable layers or modules (called adapters) into the frozen network. Only these new pieces are trained — the original pretrained weights stay frozen at their pretrained values.

EXAMPLE Adapter layers, prefix tuning

FAMILY 02

Selective Methods

Freeze most of the network but unfreeze a specific, small subset of existing parameters — for example, only bias terms, or only the final layer of each transformer block.

EXAMPLE Bias-only fine-tuning

FAMILY 03

Reparameterization Methods

Represent the weight update itself with far fewer parameters than the full weight matrix. Instead of learning a full , learn a low-rank approximation of it. belongs to this family.

EXAMPLE LoRA (covered next)

Today's deep dive → Reparameterization (LoRA), the dominant open-source approach.

LoRA: Low-Rank Adaptation

¶ 01 • THE INSIGHT

The change needed to adapt a large weight matrix to a new task tends to have a low **"intrinsic rank"** — well-approximated by the product of two much smaller matrices. The full ΔW does not need to be learned.

¶ 02 • THE MECHANISM

For a frozen weight matrix W , LoRA adds a trainable update $\Delta W = B \times A$, where A and B are small, low-rank matrices. Only A and B are trained; W remains untouched.

CORE EQUATION • LOW-RANK FACTORIZATION OF THE WEIGHT UPDATE

Frozen Pretrained		Trainable Update		Frozen Pretrained		Trainable Low-Rank Update
W_{froz}	+	ΔW_{train}	=	W_{froz}	+	$B \times A$
$(d \times k)$		delta-W		$(d \times k)$		$B: (d \times r) \quad A: (r \times k)$ <i>where r is small</i>

Forward pass: $y = W \cdot x + B \cdot A \cdot x$

LoRA's Rank Parameter (r) — Where the Savings Come From

For a weight matrix of size $d \times k$, a full update has $d \times k$ trainable parameters. LoRA instead uses two matrices, A ($r \times k$) and B ($d \times r$), where r — the rank — is a small number chosen by the practitioner (commonly 4, 8, or 16).

APPROACH 01 · FULL UPDATE

We directly update all parameters in the original weight matrix.

FORMULA

$$768 \times 768$$

TOTAL TRAINABLE PARAMETERS

589,824

APPROACH 02 · LORA (RANK $R = 8$)

We decompose the update into two small, low-rank matrices.

$$\text{Matrix A } (8 \times 768) = 6,144 \text{ params}$$

$$\text{Matrix B } (768 \times 8) = 6,144 \text{ params}$$

TOTAL TRAINABLE PARAMETERS

12,288 ($6,144 + 6,144$)

Savings Ratio: **12,288 / 589,824 $\approx$ 2.08%** of full update size (a ~98% reduction for this matrix)

Across an entire model, this routinely **reduces trainable parameters by 100 to 1000 times** — turning an out-of-reach fine-tuning job into one that fits comfortably on a single consumer GPU.

Likely Faculty Question

? Faculty Question

If **LoRA** only trains a tiny fraction of parameters, does it sacrifice model quality compared to full fine-tuning?

◆ The Answer

In practice, **LoRA** frequently **matches full fine-tuning quality** on many downstream tasks, and sometimes exceeds it.

Updating fewer parameters acts as a form of regularization, reducing the risk of catastrophic forgetting of the base model's general knowledge.

The rank r is the main quality/efficiency knob. While too low a rank can underfit, a modest rank of **8 to 16** captures the needed adaptation with a small fraction of compute cost.

⌘ **LIVE DEMO CONNECTION:** This reinforces why a target rank of $r = 8$ was chosen for today's live training demo in Part Eight.

QLoRA: Combining LoRA With Quantization

QLoRA loads the frozen base model in a heavily compressed **4-bit format**, while the small trainable **LoRA** matrices are still updated in higher precision.

This combination allows fine-tuning of genuinely large models (billions of parameters) on a single consumer-grade GPU — a direct preview of the quantization concepts covered next.

CROSS-SECTION BRIDGE

Bridges Part 03 (**LoRA**) and Part 05 (Quantization).

LAYER 01 · BASE

FROZEN BASE MODEL · 4-bit storage

Heavily compressed — minimal memory footprint

↓ Forward + backward pass combines both — gradients flow only into gold block. ↓

LAYER 02 · ADAPTER

TRAINABLE LoRA MATRICES (A, B) · higher precision

Small enough to fit in higher precision without breaking budget

Section Summary: PEFT & LoRA

01 · Core Concept

PEFT Paradigm

Reframes fine-tuning from "retrain everything" to training a small, targeted addition on top of a frozen, already-capable base model.

02 · Dominant Method

LoRA Adaptation

The leading reparameterization technique. Introduces a rank parameter r that directly controls the trade-off of trainable parameters.

03 · Efficiency

Resource Savings

A modest rank of **8-16** routinely captures most adaptation, at just **1-2%** of full fine-tuning's memory cost.

→ **TRANSITION**

We now turn to the second half of the resource-optimization story: once a model is trained, how do we shrink and speed it up for deployment?

Precision Formats

FP32, FP16, BF16, and INT8: what these numeric formats are, and what changes when a model moves between them.

04

FP32

Single Precision
32-bit Floating Point

FP16

Half Precision
16-bit Floating Point

BF16

Brain Float
16-bit Alternate Range

INT8

Integer Format
8-bit Fixed Precision

Why Numeric Precision Is a Resource Lever

01 · THE BASIC UNIT

Every **weight, activation, and gradient** in a neural network is stored as a number. The format chosen directly determines how many bytes each number takes, how much memory the model needs, and how fast hardware can process it.

02 · THE 32-BIT DEFAULT

Standard training historically used 32-bit floating point (**FP32**). Modern efficiency techniques ask: **do we need all 32 bits, everywhere, all the time?** Once you accept that the answer is no, optimization opens up.

DEFAULT **FP32**



HALF PRECISION **FP16 · BF16**



QUANTIZED **INT8**

"The optimization stack opens up once you accept fewer bits than 32."

Comparing the Common Formats

Four numeric formats you'll encounter across modern deep learning workflows.

Format	Bits	Typical Use
FP32 (single precision)	32	Historical training default; highest precision, largest memory footprint.
FP16 (half precision)	16	Halves memory vs. FP32; narrower numeric range, can underflow/overflow.
BF16 (bfloat16)	16	Same memory savings as FP16, but keeps FP32's wide exponent range, more stable for training.
INT8 (8-bit integer)	8	Quarter the memory of FP32; used mainly for inference after quantization, not raw training.

◆ Memory savings compound across billions of parameters — **FP16** cuts a 24 GB model to ~12 GB; **INT8** cuts it to ~6 GB.

FP16 vs. BF16 — Why the Distinction Matters

01 · FP16 (MORE PRECISION)

Narrower Range, Higher Precision

Allocates 10 bits to the mantissa and 5 to the exponent. Its narrow range makes very large or small gradient values prone to overflow or underflow to zero, requiring "loss scaling" tricks to stabilize training.

02 · BF16 (WIDER RANGE)

FP32-Equivalent Range, Less Precision

Allocates 8 bits to the exponent and 7 to the mantissa. Because it shares FP32's wide dynamic range, it avoids stability issues without needing tricks, making it the modern training default.

BIT-LEVEL FORMAT COMPARISON

FP16 · 1 Sign, 5 Exponent, 10 Mantissa



0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

More precision · narrower dynamic range

BF16 · 1 Sign, 8 Exponent, 7 Mantissa



0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Less precision · FP32-equivalent range

☐ Sign
 ☒ Exponent
 ☐ Mantissa

BF16 provides FP32-equivalent dynamic range with half the physical footprint.

Mixed-Precision Training

The standard recipe for getting **FP16/BF16** speed and memory gains without losing **FP32** accuracy.

Step 01 Keep a Master Copy

FP32 Master Weights

Maintain a master copy of all weights in FP32. This accumulates small updates accurately over many training steps, avoiding precision loss that would otherwise compound across thousands of gradient steps.

Step 02 Compute in Low Precision

FP16 / BF16 Forward & Backward

Run the forward and backward pass in FP16 or BF16. Most of the actual computation runs in the smaller format, gaining speed and memory savings where it matters most inside the matmul kernels.

Step 03 Update Master Copy

Update FP32 Master Weights

Update the FP32 master weights from the lower-precision gradients. The accurate FP32 copy is then re-cast to FP16/BF16 for the next step so the small precision errors never accumulate into large drift.

THE TRAINING LOOP

FP32 master weights

CAST

FP16 forward / backward

GRADIENTS

FP32 weight update

CAST (LOOP)

The **FP32 master** anchors accuracy; **FP16/BF16** handles speed. Loop repeats every step.

Likely Faculty Question

01 · FACULTY QUESTION

If lower precision loses information, why doesn't it noticeably hurt model accuracy?

02 · ANSWER & ANALYSIS

Neural networks are highly tolerant of small numeric noise — a well-documented empirical finding. The gradient signal driving learning is typically far larger than the rounding error introduced by dropping formats.

Because the overall signal dominates, the network still learns the same function when moving from **32 to 16 bits**. This inherent robustness is precisely what makes quantization to even lower formats (**8-bit and below**) viable for inference.

→ **TRANSITION** Sets up the next section — pushing precision all the way down to **8-bit** and **4-bit** representations.

Section Summary: Precision Formats

Numeric format is the first and simplest resource lever: moving from **FP32** to **FP16/BF16** **roughly halves memory** and often meaningfully speeds up computation, with minimal accuracy impact for most tasks. **BF16** is the modern training default — same memory savings as **FP16** without the range headaches.

MEMORY

$\sim 1/2\times$

FP16/BF16 vs FP32

TRAINING DEFAULT

BF16

FP32-equivalent range, fewer overflow headaches

ACCURACY COST

Minimal

Networks tolerate small numeric noise

→ TRANSITION

Quantization takes this same idea further, compressing to **8-bit** or even **4-bit** representations, with a more deliberate process to preserve accuracy at that scale.

Quantization

Compressing a trained model's weights to lower-bit representations, and the two dominant strategies for doing it well.

05

01 • TARGET FORMATS

INT8 / 4-bit

Compressing weights down to highly efficient low-bit integer representations.

02 • STRATEGIES

PTQ vs QAT

Choosing between fast Post-Training Quantization or accuracy-preserving Quantization-Aware Training.

03 • ECOSYSTEM

bitsandbytes · GPTQ · ONNX

Leveraging modern libraries and formats to deploy optimized models on target hardware.

What Quantization Actually Does

Quantization maps a wide range of floating-point values to a much smaller set of discrete levels — typically 8 bits (256 levels) or even 4 bits (16 levels) — using a **scale** and **zero-point** to convert between the original and compressed ranges.

INT8 PERFORMANCE ADVANTAGE

~1/4X model size

Integer arithmetic is generally faster than floating-point arithmetic, delivering significant speedups on target hardware.

FP32 · CONTINUOUS VALUES

4 bytes each



↓ scale + zero-point

INT8 · 256 LEVELS

1 byte each



Each FP32 value maps to the nearest INT8 level — controlled, reversible approximation to yield massive space savings.

Post-Training Quantization vs. Quantization-Aware Training

Two strategies for when and how quantization is applied to a model.

Approach	When It Happens	Trade-off
Approach A Post-Training Quantization PTQ	Applied after a model is fully trained, using a small calibration dataset to determine scale factors.	Fast and simple, but can lose more accuracy on quantization-sensitive models.
Approach B Quantization-Aware Training QAT	Quantization effects are simulated during training itself — the model learns to be robust to rounding.	Better accuracy retention, but requires retraining, more time and compute cost.

▶ Today's live demo uses **PTQ** — fast and the right choice for a 50-minute demonstration window.

How PTQ Actually Calibrates a Model

A small, representative batch of real input data is passed through the trained model. The actual range of activation and weight values observed is used to compute a **scale factor** — the conversion ratio between the FP32 numeric range seen in practice and the INT8 target range.

Without observing realistic value ranges, the scale factor could badly mismatch the model's actual behavior — wasting precision on values that never occur, or clipping values that do. Calibration is the step that makes **PTQ** “know” what your model actually does.

THE PTQ CALIBRATION PIPELINE

01



Trained FP32 Model

Baseline high-precision model ready for quantization mapping.



02



Calibration Run

Pass a representative calibration batch through the network.



03



Observe Ranges

Analyze and record activation and weight value distributions.



04



Compute Scaling

Generate final scale and zero-point conversion matrices.

Common Quantization Tooling

The three libraries you'll encounter most often in modern **PyTorch** workflows.



HUGGING FACE ECOSYSTEM

bitsandbytes

Widely used with Hugging Face Transformers for **8-bit** and **4-bit** loading, including the format used inside **QLoRA**.

The default choice for low-bit LLM loading in the open-source community.



LLM-SPECIFIC

GPTQ

A post-training quantization method specifically designed for large language models.

Minimizes accuracy loss at very low bit-widths (down to **4-bit** or lower) using a calibration-based approach.



DEPLOYMENT ECOSYSTEM

ONNX Runtime

Built into the **ONNX** ecosystem. Quantizes a model as part of preparing it for deployment across multiple hardware targets.

Pairs naturally with the **ONNX** export flow covered in Part Six.

i Other formats like **NF4** (used inside **QLoRA**) round out the low-bit landscape for LLMs.

Likely Faculty Question

❓ FACULTY QUESTION

How low can you quantize before accuracy meaningfully degrades?

📖 Quantization Thresholds

8-Bit Quantization: Now routine with minimal measurable accuracy loss for most models.

4-Bit Quantization: Viable for large language models specifically because their sheer parameter count provides redundancy that tolerates aggressive compression (using formats like **GPTQ** or **NF4**).

⚠️ Sensitivity & Validation

Model Scale Dependencies: Very small models, or models already operating near their capacity limit, tend to be significantly more sensitive to precision reduction.

Mandatory Benchmarking: Real-world validation after quantization—rather than assuming it is safe—is standard practice to avoid silent quality degradation.

→ *Directly motivates the live benchmarking in Part Eight.*

A Real Gotcha Worth Knowing: Not All Models Quantize the Same Way



GOTCHA Silent failure mode. Code runs without error, but quantization has no effect.

01 / THE QUIRK

GPT-2 & Conv1D

GPT-2 (used in today's demo) uses `Conv1D` layers internally for attention and feed-forward weights instead of the standard `nn.Linear`.

02 / THE SILENT SKIP

Library Limits

Most tools only target `nn.Linear` layers. They will **silently skip** `Conv1D` ones, executing "successfully" without actually quantizing anything.

03 / THE SOLUTION

Transposition

Transpose the `Conv1D` weights into equivalent `nn.Linear` containers before quantizing. Output difference is pure noise ($\sim 1e-4$).

TEACHING POINT

"Quantization ran with no errors" is not the same as "quantization actually changed anything." Always verify the effect, not just that the code executed successfully.

A Second Gotcha: Measuring Quantized Model Size Correctly



GOTCHA · A trusted measurement tool reports zero reduction after working quantization.

01 / THE BUG

False Alarm

Hugging Face's

`get_memory_footprint` can report zero size reduction after quantization, even when the model actually quantized correctly.

02 / THE ROOT CAUSE

Facade Effect

Quantized formats (from `torchao`) expose a backward-compatible "facade" reporting original `FP32` size to legacy code, fooling standard tools.

03 / THE FIX

Direct Probe

Measure the actual underlying compressed storage tensors directly, rather than relying on high-level size-reporting APIs.

TEACHING POINT

A surprising benchmark result is a reason to double-check your measurement method before concluding a technique failed.

Section Summary: Quantization

Quantization compresses a trained model's numeric representation, trading a controlled amount of precision for **gains in memory footprint and inference speed** — but only if you verify it actually applied correctly and measure it correctly.

01 / MEMORY FOOTPRINT

$\sim \frac{1}{4} \times$

INT8 vs FP32 original

02 / TWO STRATEGIES

PTQ • **QAT**

Demo uses **PTQ** — fast, calibration-driven

03 / VERIFICATION

Mandatory

“No errors” ≠ “it worked”

TRANSITION

*A quantized model still needs a way to run efficiently across different hardware and deployment targets, which is what **ONNX** addresses next.*

Model Export & ONNX

Taking a trained model out of its training framework and into a portable, hardware-optimized format for deployment.

06

01 / DEVELOPMENT

✦ PyTorch

Train or fine-tune your model in PyTorch, establishing the primary weights and computation graph before porting.

02 / PORTABILITY

▮ .onnx Export

Trace and serialize the computation graph into a standard, framework-independent file format.

03 / DEPLOYMENT

➤ ONNX Runtime

Execute high-performance inference on optimized target runtimes without any legacy framework dependencies.

The Deployment Problem ONNX Solves

01 / THE PYTHON DEPENDENCY PROBLEM

A model trained in **PyTorch** is, by default, tied to a Python process with PyTorch installed. Many real deployment targets — mobile apps, browsers, embedded devices, and production servers — cannot or should not carry that full dependency.

02 / THE ONNX INTEROPERABILITY SOLUTION

ONNX (Open Neural Network Exchange) is a standard, framework-independent file format for representing a trained model's computation graph and weights. Once exported, it can be run by any compatible runtime.



The Export & Deployment Pipeline

Three steps from “trained model” to “deployed, optimized inference.”

01 / DEVELOPMENT

▣ PyTorch

Train or fine-tune in your framework of choice (PyTorch, TensorFlow). Covers everything from training to parameter-efficient fine-tuning (**PEFT**, **LoRA**) and quantization.



02 / PORTABILITY

▣ .onnx Export

Trace and serialize the computation graph and weights into the standard, framework-independent **.onnx** format. The model leaves its training framework behind.



03 / DEPLOYMENT

➤ ONNX Runtime

Execute high-performance inference using a lightweight, highly optimized engine. Often runs significantly faster and deploys without the training framework installed.

Why ONNX Runtime Is Often Faster

Three independent optimization layers — they stack.



OPTIMIZATION 01

Graph Optimization

Redundant or fusable operations in the computation graph are automatically simplified or merged before execution.

Fewer kernel launches, less overhead, same outputs.



OPTIMIZATION 02

Hardware-Specific Execution

ONNX Runtime can dispatch operations to hardware-specific accelerators via execution providers — **CUDA**, **TensorRT**, **OpenVINO**, **CoreML**, and others — picking the fastest path.



OPTIMIZATION 03

Combines With Quantization

ONNX models can themselves be quantized, stacking graph optimization and reduced-precision inference. Size, speed, and portability gains compose multiplicatively.

✓ VERIFIED LIVE

In today's live demo — **ONNX Runtime** was the fastest option tested, faster than **FP32 PyTorch** and faster than **INT8** quantized.

Likely Faculty Question

FACULTY QUESTION

If a model already runs fine in **PyTorch**, what's the concrete benefit of exporting to **ONNX**?

BENEFIT 01

Portability

Runs without **PyTorch** installed, enabling cross-platform integration in other languages via native bindings.

BENEFIT 02

Inference Speed

ONNX Runtime's graph optimizations frequently outperform native framework inference, with zero retraining required.

BENEFIT 03

Standard Interface

The same **.onnx** file can target very different hardware backends through various execution providers.

✓ VERIFIED LIVE

Benefit #2 is verified live in Part Eight — **ONNX Runtime** beats both **FP32** and **INT8** on inference speed.

Important Clarification: ONNX Export Does Not Shrink a Model by Itself

✗ COMMON MISCONCEPTION

Exporting to **ONNX** reduces model size the way quantization does.

✓ REALITY

ONNX changes execution format & runtime, not numeric precision or parameter count. Size stays identical to original **FP32** PyTorch model.

EXPORTED ONNX MODEL

475 MB

Exported **ONNX** model size (today's live demo)

ORIGINAL PYTORCH MODEL

474.7 MB

Original **FP32** PyTorch model size (today's live demo)

*Essentially the same size — but the **ONNX** version was the fastest of every version tested.*



Size reduction from **ONNX** only comes if you additionally quantize the **ONNX** model — a separate, optional step covered in today's demo.

Section Summary: Model Export & ONNX

ONNX decouples a trained model from its original training framework, enabling **portable, optimized, and often faster inference** across deployment targets — independent of, and complementary to, quantization.

DECOUPLING

Framework-free

.onnx runs entirely independent of **PyTorch** or other training frameworks.

SPEED

Faster

Leverages built-in graph optimizations and hardware execution providers.

COMPOSABLE

+ Quantization

Acts as an independent, complementary performance lever alongside quantization.

TRANSITION

We've now covered the full resource-optimization stack: **PEFT** and **LoRA** for efficient training, precision formats and quantization for efficient storage and compute, and **ONNX** for efficient deployment. Time to see all of it applied together, live.

Live Demo: Two Models, One Journey

From a tiny transformer trained from scratch, to a real pretrained model fine-tuned with LoRA, quantized, and exported.

07



MODEL 01 • ~28K PARAMS

Tiny Scratch Transformer

A character-level transformer trained live on toy data to make deep learning mechanics transparent.

- Watch loss convergence in real time
- Inspect full training loop mechanics
- Ideal for understanding the math end-to-end



MODEL 02 • GPT-2 • 124M PARAMS

Pretrained GPT-2 Small

A production-grade model optimized using the full state-of-the-art resource stack.

- Fine-tune with parameter-efficient **LoRA**
- Compress weights using **INT8 Quantization**
- Export to high-performance **ONNX Runtime**

What We're About to Build

Two models, in sequence, demonstrating the full optimization stack end to end.

MODEL 01 · ~28K PARAMETERS

Tiny Scratch Transformer

Trained live on toy text data. Designed to make the full training loop mechanics completely transparent and watchable in real time.

✓ WATCHED CONVERGE

MODEL 02 · 124M PARAMETERS

Pretrained GPT-2 Small

A real, production-grade model optimized using the full state-of-the-art resource stack.

- Fine-tune with parameter-efficient **LoRA**
- Compress weights using **INT8 Quantization**
- Export to high-performance **ONNX Runtime**
- Conduct direct head-to-head **Benchmark**

*Same model. Four versions. **Direct head-to-head comparison.***

What Each Segment Demonstrates

Each segment of the demo maps back to a specific concept from Part A.

SEGMENT 01

Training Loop Mechanics

Forward pass, loss computation, backward pass, optimizer step — the fundamental loop underneath every model in this session, made visible on something small enough to inspect directly.

→ MAPS TO → PART 02 · COST OF SCALE

SEGMENT 02

PEFT and LoRA in Practice

Loading a frozen pretrained model, attaching small trainable LoRA layers, and fine-tuning only those — with a live parameter-count comparison showing 0.24% trainable.

→ MAPS TO → PART 03 · PEFT & LORA

SEGMENT 03

Quantization & Export

Taking the fine-tuned model through INT8 quantization and ONNX export, then directly comparing file size and inference speed before and after — with the two gotchas from Part 05 fixed along the way.

→ MAPS TO → PART 05 & PART 06

A Note on Framework Choice

¶ 01 — TODAY'S STACK

Today's hands-on notebook uses **PyTorch**, alongside the **Hugging Face** ecosystem: **transformers**, **peft**, and **ONNX** export tooling.

¶ 02 — WHY THIS STACK, NOT A STATEMENT

This isn't a statement that **PyTorch** is categorically better; **Keras** and **TensorFlow** have their own robust tooling. PyTorch is used here because PEFT, LoRA, and ONNX tooling is overwhelmingly built **PyTorch-first**.

PyTorch + Hugging Face

✓ USED TODAY

- **transformers** — State-of-the-art NLP models
- **peft** — Parameter-Efficient Fine-Tuning
- **bitsandbytes** — 8-bit & 4-bit quantization
- **onnxruntime** — High-performance inference

TensorFlow + Keras

✦ EQUALLY CAPABLE

- **Keras Mixed Precision** — Float16 optimization
- **TFLite** — Mobile & edge deployment
- **TF-TRT** — TensorRT integration

Choose the ecosystem with the strongest community documentation for your specific task.

GPT-2's Actual Scale, Confirmed Live



VERIFIED LIVE · COUNTED IN TODAY'S SESSION

■ TINY FROM-SCRATCH MODEL

~28,000

parameters

Character-level transformer trained live in Segment 01.

28K

◆ GPT-2 (SMALL)

124,439,808

parameters · confirmed by count

Not an approximation — counted in today's live session via

`model.parameters()`.

124M

$124,439,808 / 28,000 \approx \sim 4,400\times$ more parameters

A concrete sense of scale between "toy model" and "real pretrained model."

Live Results, Benchmarking & Takeaways

Every number below is a real, measured result from this exact session's live demo run, not an illustrative estimate.

✓ VERIFIED LIVE / ◆ SIZE · SPEED · QUALITY / ◆ MEASURED TOGETHER

08

Live Results — Fine-Tuning



VERIFIED LIVE · MEASURED IN TODAY'S SESSION

Metric	Value
GPT-2 total parameters	124,439,808
LoRA trainable parameters (rank r=8)	294,912
Trainable percentage	0.24%
LoRA target module	<code>c_attn</code> (GPT-2's combined attention projection)

0.24%
TRAINABLE

Only 0.24% of the model's parameters were updated during fine-tuning. This is the concrete, measured proof of parameter-efficient fine-tuning discussed in [Part Three](#).

Live Results — Quantization

✓ VERIFIED LIVE · MEASURED IN TODAY'S SESSION

Version	Size	Notes
FP32 (Linear-converted)	474.70 MB	Baseline
INT8 (quantized)	269.14 MB	43.3% smaller

43.3%

SMALLER

This required the **Conv1D** to **nn.Linear** conversion and the corrected measurement approach covered in **Part Five**. The naive path would have shown zero reduction.

The Surprising Result: Quantization Made the Model Smaller, But Slower

⚠️ **SURPRISING FINDING** — real trade-off, not a mistake. Measure both axes, always.

FP32 BASELINE ✓ VERIFIED

~0.99 sec

average generation time



INT8 QUANTIZED ✓ VERIFIED

~3.20 sec

avg time · ~3× slower 43% smaller · but slower

This is a genuine, real trade-off, not a mistake: the fused low-level kernel that makes **INT8** matrix multiplication fast is a newer, less mature code path on this hardware than **FP32**'s well-optimized path.

Speed gains from quantization are hardware- and kernel-maturity-dependent, while the memory savings are consistent and reliable.

TEACHING POINT: "Never assume a technique that helps on one axis (size) automatically helps on every axis (speed). Always measure both."

Live Results — ONNX Export



VERIFIED LIVE · MEASURED IN TODAY'S SESSION

Version	Size	Avg. Generation Time
ONNX Runtime	475.09 MB	~0.80 sec ⚡ FASTEST OF ALL THREE

~0.80 sec
FASTEST

ONNX Runtime was the fastest option tested overall — faster than plain FP32 PyTorch and faster than the quantized version — purely from graph-level runtime optimizations, without any change in model size or numeric precision.

Full Comparison — Every Version Tested Today

✓ VERIFIED LIVE · MEASURED IN TODAY'S SESSION

Version	Size (MB)	Avg. Generation Time (s)	What it won	What it lost
FP32 (Linear-converted)	474.70	0.99	🚩 Baseline	—
INT8 Quantized	269.14	3.20	⬆️ 43% smaller	↘️ 3×+ slower
ONNX Runtime	475.09	0.80	⚡ Fastest overall	ⓘ No size reduction alone

KEY LINE

No single technique won on every axis. This is a live demonstration of the core principle of this entire session: *always measure size, speed, and quality together*. Never assume a technique that helps on one dimension helps on all of them.

What We Actually Hit While Building This Demo

OPTIONAL · TIME-PERMITTING

Three real ecosystem issues — exactly the kind of friction you'll hit doing this yourself.

⊗ ISSUE 01

GPT-2 `Conv1D` layers break naive quantization

Fixed with a verified weight conversion step — transpose `Conv1D` weights into `nn.Linear` before quantizing.

Covered in Part Five, Gotcha #1.

⊗ ISSUE 02

`get_memory_footprint` reported zero reduction

Fixed by measuring actual compressed storage tensors directly, rather than trusting high-level size-reporting methods.

Covered in Part Five, Gotcha #2.

⊗ ISSUE 03

Dependency version check caused import error

Triggered in code not using `torchao`. Fixed by installing correct `torchao` before importing `peft`.

Environment Setup Issue

FRAMING These aren't failures to hide — they're exactly the kind of **real, current ecosystem friction** you'll hit doing this yourself, and now you know what to look for.

The Three Big Ideas From Today

If you remember three things from this session, remember these.

BIG IDEA 01



Train Less, Not From Scratch

PEFT and **LoRA** freeze a capable pretrained model and adapt it with a small number of new parameters — democratizing fine-tuning beyond large compute budgets.

TODAY'S DEMO

0.24% of parameters trained, full task adaptation achieved.

BIG IDEA 02



Store and Compute Smaller, Carefully

Precision formats and quantization reduce bytes and operations per parameter, with real trade-offs that must be measured, not assumed.

TODAY'S DEMO

43% size reduction — but **3×** slower inference, a real trade-off.

BIG IDEA 03



Deploy Portably and Fast

ONNX decouples a trained model from its training framework, enabling optimized inference across diverse real-world deployment targets.

TODAY'S DEMO

ONNX Runtime was the fastest version tested, with no size change.

Mapping This Back to Your Own Research

¶ 01 — TECHNIQUES THAT APPLY DIRECTLY

If you're working with any pretrained model — vision, language, or otherwise — and compute or deployment resources are a constraint, every technique covered today applies directly: **LoRA** for efficient adaptation, quantization for a smaller footprint measured correctly, **ONNX** for portable deployment.

¶ 02 — BRING IT TO THE OPEN DISCUSSION

The open discussion that follows is a chance to connect these techniques to constraints already present in your own work — a specific model you'd like to fine-tune, a deployment target you need to hit, or a compute budget you need to fit within.

WHERE THE TECHNIQUES APPLY

VISION MODELS 🕒

LoRA fine-tune **CLIP** / **ViT** · quantize for edge deployment

LANGUAGE MODELS 🕒

LoRA fine-tune **LLMs** · **INT8**/4-bit for inference · **ONNX** for serving

SPEECH / AUDIO 📺

LoRA Whisper variants · **ONNX** for real-time inference

Open Discussion

⦿ FLOOR IS OPEN

*Where in your own research or teaching would a **resource-constrained model** — smaller, faster, or cheaper to run — open up something that's currently out of reach?*



A specific model you want to fine-tune?



A deployment target you need to hit?



A compute or memory budget to fit within?

⦿ ~10 MIN · OPEN FLOOR

Thank You

Deep Learning Applications for Resource Optimization

Efficient training, compression, and deployment of neural networks



Consulting

Er. Abhishek Kumar Shukla

Senior AI Developer

UniConverge Technologies Pvt. Ltd.

